Agentic LLMs as Co-Designers for Open Networking Switch Platforms

Technical Whitepaper

Giridhar.N

Table of Contents

Introduction 2
Problem and Background 3
Manual platform bring-up in open networking NOS 3
Impact of AI/ML and high-speed fabrics 3
Solution Overview: Agentic LLM as Co-Designer 4
Architecture 5
Overview 5
Agentic flow and tools 6
Platform abstraction outputs 6
Proof-of-Concept 7
LLM designers in the PoC engine 7
Synthetic datasets 8
Design workbench and backend flow 9
GitHub Repo 11
From Abstractions to Code Templates 11
Future directions 12
Conclusion 12

Introduction

Bringing a new hardware switch platform into an open networking network operating system (NOS) such as SONiC remains a highly manual and error-prone process. Engineers must interpret ASIC documentation, board pin-maps, and platform datasheets, then translate these artifacts into platform descriptions, port/SerDes mappings, and SDK configuration profiles that the NOS can consume. This is especially painful in modern data centers and AI fabrics, which rely on high-speed ports, complex breakout modes, and advanced traffic features, and where reducing platform bring-up time is critical to overall time-to-market.

This whitepaper proposes and demonstrates a proof-of-concept (PoC) in which a large language model (LLM) acts as an agentic co-designer for open networking data center leaf and spine switch platforms, with an emphasis on AI and cloud-class fabrics. Instead of starting from low-level configuration files, a designer provides a natural-language description of a platform ASIC family, port matrix, form factors, breakout behavior, and intended NOS role. An agentic backend then orchestrates tools and retrieval-augmented generation (LLM + RAG over structured artifacts) using synthetic CSVs and parsed documents without a dedicated vector database in this PoC to parse inputs, normalize them into structured data, validate them against SONiC-style constraints, and synthesize a structured platform abstraction.

To avoid reliance on NDA-protected materials, the PoC uses synthetic but internally consistent datasets that emulate realistic top-of-rack (ToR) and spine platforms.
A browser-based design workbench and backend orchestrator demonstrate an end-to-end flow from prompt to platform abstraction. The same pattern can be extended by enterprises and open networking communities with access to proprietary datasets and licensing.

Problem and Background

Manual platform bring-up in open networking NOS

In open networking stacks such as SONiC, supporting a new hardware platform typically follows a multi-step "porting" process.
Engineers must:

- Review ASIC data sheets and programming guides.
- Study platform datasheets to understand port counts, form factors, breakout capabilities, and supported optics.
- Consult board-level documentation for SerDes lane assignments, pin-maps, and power/thermal constraints.
- Encode this knowledge into platform definitions, port mapping files, and ASIC/NOS configuration.

These steps are repeated for each new ToR, leaf, spine, or AI fabric device. While the specifics differ per platform, the pattern of work mapping human-readable documents to structured NOS configuration remains largely the same.

Impact of AI/ML and high-speed fabrics

Modern data centers and AI/ML clusters increase the complexity of this process. Platforms may expose dozens of 100G, 400G, or 800G ports, each with multiple breakout modes and the need to support advanced features such as RDMA, VXLAN overlays, and adaptive routing for GPU fabrics.

Enterprise SONiC-style distributions add AI-focused capabilities (for example, RoCEv2, dynamic load balancing, and enhanced ECMP hashing) and explicitly target high-performance silicon generations.
As a result:

- Port and lane mapping mistakes become more costly, because they interact with increasingly complex traffic features and deployment constraints.
- Bringing up each new platform requires repeated, largely similar reasoning across documents and internal configuration standards.

The combination of manual effort, higher complexity, and tight project schedules creates a natural opportunity for automation and assistive tooling.

Solution Overview: Agentic LLM as Co-Designer

The proposed approach is to treat a large language model as a co-designer for open networking switches, spanning platform, software, and datapath concerns, rather than just a one-shot text generator. Instead of editing low-level configuration files directly, a user expresses platform intent in natural language, and it is embedded in an agentic workflow that coordinates a sequence of tools and intermediate representations.

For example:
"Design a 1U top-of-rack switch with 32 high-speed ports suitable for AI workloads. Each port should support both full-rate operation and 8-way breakout. Target an open networking *NOS similar to SONiC and assume a modern merchant-silicon ASIC family."*

The agent:

1. Parses relevant documents and structured data.
2. Normalizes information into machine-readable tables and schemas.
3. Validates the information against NOS-style constraints and patterns.
4. Synthesizes a platform abstraction that existing porting flows can consume.

Architecture

Overview

- A browser-based design workbench, where the user enters platform prompts, triggers design runs, and inspects generated outputs.
- An agentic LLM orchestrator, which hosts prompt templates, coordinates tool calls, and assembles final artifacts.
- A hardware knowledge and tools layer, powered by synthetic datasets and generic parsers/validators.

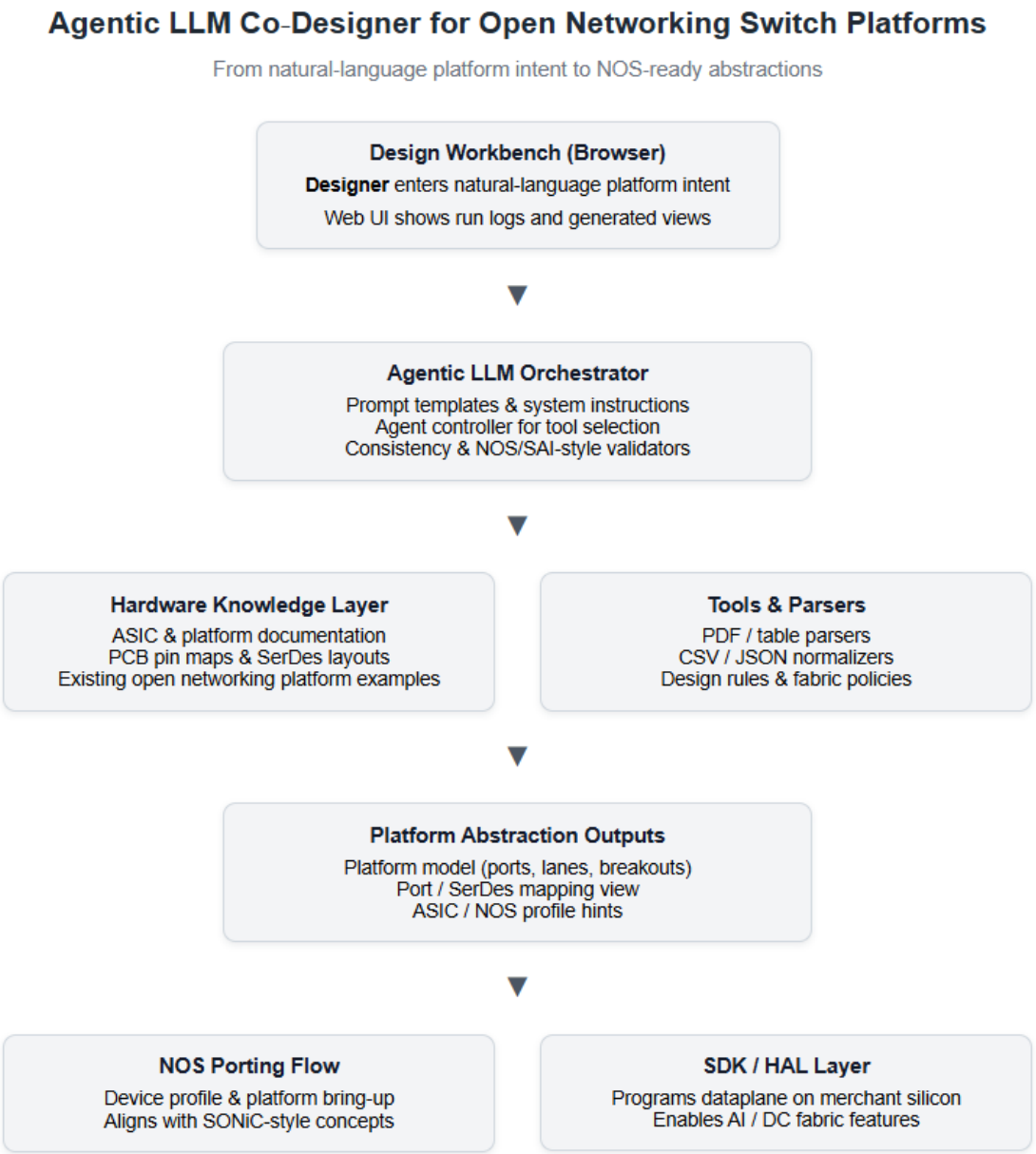

Figure-1: Agentic LLM co-designer Architecture

- Figure-1 illustrates the overall agentic LLM co-designer architecture, from the Design Workbench to the orchestrator and hardware knowledge layer.

Agentic flow and tools

The agentic orchestration layer combines model reasoning with concrete tools:

- **Parsers:** Ingest public datasheets and example platform materials, extracting attributes such as port counts, form factors, breakout capabilities, and management interfaces.
- **Normalizers:** Convert these inputs into consistent CSV/JSON tables that capture ports, SerDes lanes, speeds, and other attributes.
- **Validators** enforce constraints inspired by SONiC documentation and typical AI-fabric feature sets, such as valid speed combinations, breakout patterns, and resource limits.

The LLM uses these tools as external reasoning steps, calling them as needed and integrating their results into a coherent platform model.

Platform abstraction outputs

From the validated model, the agent produces structured outputs that are conceptually aligned with what open networking NOSes expect:

- A platform description, capturing ports, lanes, form factors, and breakout modes.
- A port/SerDes mapping view, describing logical ports, lane groups, and nominal speeds.
- A set of ASIC/NOS “profile hints”, indicating expected capabilities, AI-fabric features, and configuration defaults.

These outputs can be mapped into concrete artifacts (for example, platform definition files, port maps, and profile configurations) in a SONiC-style porting process without changing the downstream tooling.

Proof-of-Concept

LLM designers in the PoC engine

The PoC backend organizes the agent into a small set of co-designer components, each backed by LLM prompts and RAG over synthetic platform data.

- Platform designer
  - Focuses on the physical platform view. From the user's prompt and selected synthetic datasets, it proposes a SONiC-style platform abstraction: ports, lanes, breakout modes, form factors, and high-level ASIC profile hints. Its output is an internal platform model that downstream designers consume.
- Software designer
  - Works from the platform model to suggest NOS-facing artifacts: platform description fragments, initial port configuration tables, and profile defaults that align with SONiC-style porting guidance. It keeps proposals within the constraints validated earlier by the platform designer.
- Datapath designer
  - Interprets the same platform model from a traffic and AI-fabric perspective, emphasizing how ports, speeds, and breakout options map to features such as ECMP, VXLAN, or RDMA-capable fabrics. In the PoC, this is primarily descriptive, feeding hints that can later guide QoS and fabric configuration templates.
- Master designer
  - Coordinates the sequence, receives the user prompt from the workbench, invokes the platform designer to build and validate the core model, then calls the software and datapath designers to enrich that model with NOS and fabric-oriented views. Finally, it assembles SONiC-compatible outputs for the UI (platform abstraction view, port/SerDes mapping, and profile hints) and optional code templates.
  - Operationally, the master designer orchestrates a series of RAG-augmented calls to the platform, software, and datapath designers, each operating on the same evolving platform model.

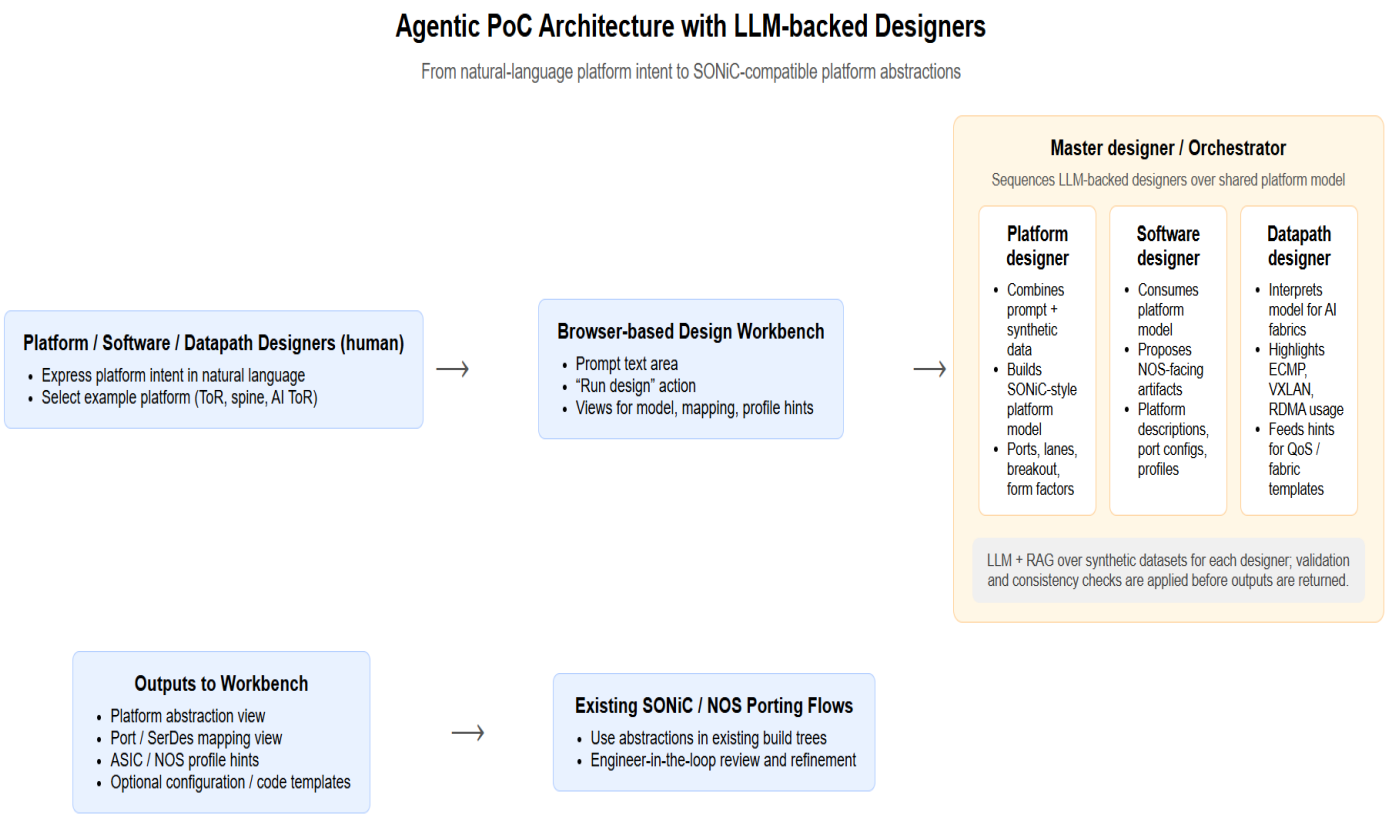

Figure-2: Agentic LLM co-designer PoC Flow

- Together, the platform, software, and datapath designers act as LLM-backed co-designers that collaboratively shape a single, SONiC-compatible platform model across the design flow.
- Figure-2 illustrates the PoC flow, including the user's platform prompt, the sequence of LLM co-designers invoked by the master designer, and the SONiC-compatible outputs returned to the workbench.

Synthetic datasets

Real PCB pin-maps, detailed ASIC programming guides, and certain platform documents are usually protected by non-disclosure agreements and licensing. To demonstrate the agentic flow without exposing proprietary content, the PoC uses synthetic datasets that emulate realistic platforms. In a production environment, the same tooling pattern is intended to ingest real vendor datasheets and board exports using industry-standard parsers (for example, PDF/tabular extractors such as pdfplumber or tabula-style tools, EDA/PCB export utilities, and internal BOM/Excel converters), normalize them into structured records as a training and grounding corpus, and feed those records into the retrieval-augmented (RAG) layer that conditions the LLM's decisions for each new platform. These datasets:

- Are constructed from publicly available information about data center switches and high-speed merchant silicon, capturing representative attributes such as port counts, breakout capabilities, SerDes lane groupings, and form factors sufficient to emulate realistic platform behavior without reproducing any proprietary or NDA-protected design details.
- Capture plausible mappings between serialized ports and physical lanes, along with formfactors and speed options, but avoid reproducing confidential designs.

Synthetic CSV files represent, for example:

- Breakout mappings for 100G, 400G, and 800G ports (for example, 1×100G to 4×25G, 1×400G to 4×100G, 1×800G to 8×100G).
- Board-level pinmaps that connect high-speed connectors (QSFP/OSFP) to ASIC SerDes lanes, along with selected control and management signals.
- ASIC pin descriptions for high-speed SerDes groups and management interfaces.

Taken together, these synthetic tables emulate realistic top-of-rack, spine, and AI-oriented platforms without reproducing any specific commercial design.

Design workbench and backend flow

The Design Workbench sits in front of the agentic orchestrator and provides a minimal UI for launching and inspecting co-designed platforms.

It is a simple web application that exposes:

- A text area where the user enters a natural-language platform prompt.
- A “Run design” control to trigger a design session.
- A set of views (tabs or panels) that display the resulting platform model, port/SerDes mapping, and ASIC/NOS profile hints returned by the backend orchestrator.

When the user triggers a run:

- The frontend sends the prompt to the backend orchestrator.
- The orchestrator uses the LLM and tools to load the appropriate synthetic CSV files, parse them, and build an internal platform model.
- The agent applies validation rules, adjusting or flagging inconsistencies.
- The final platform abstraction and auxiliary views are returned to the frontend and rendered for inspection.
- The following figures illustrate the Design Workbench: entering a natural-language platform prompt, triggering a design run, and inspecting the generated platform model, port/SerDes mapping, and profile hints.

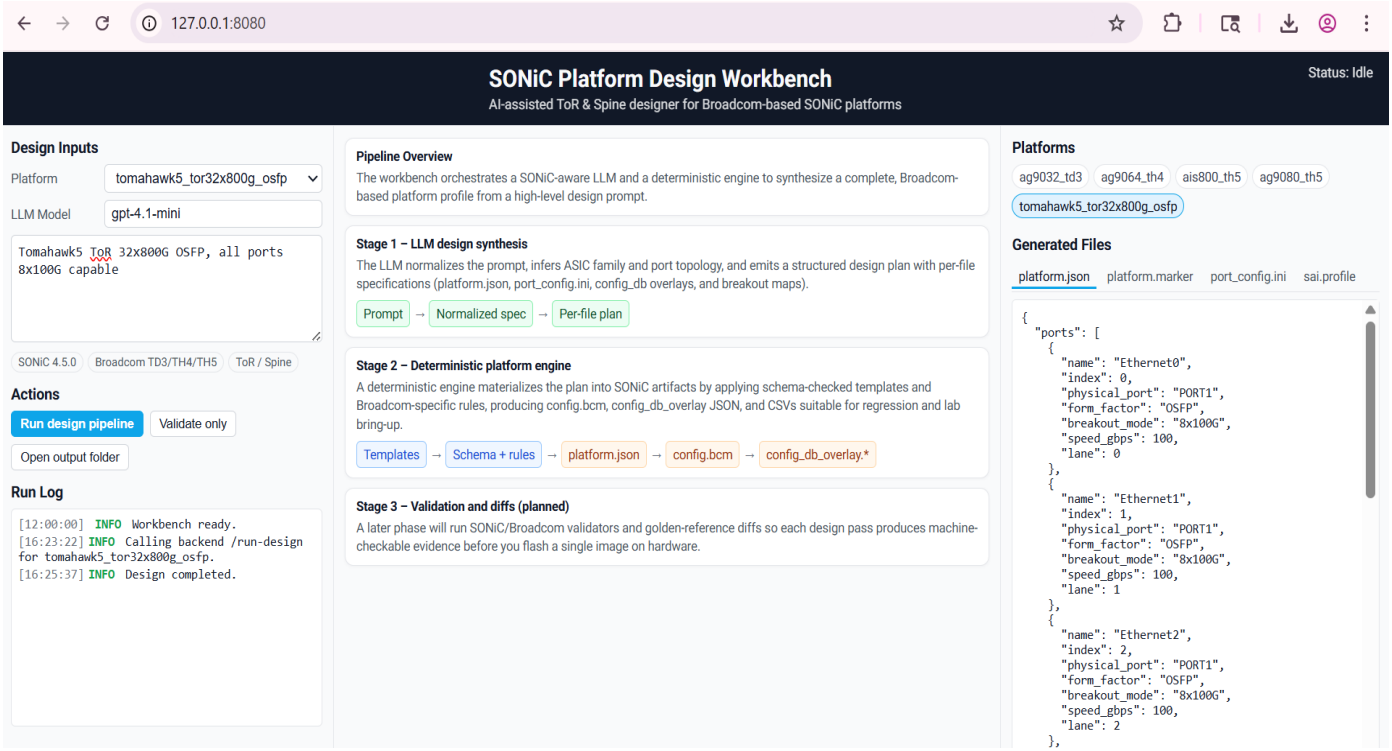

Figure-3: Agentic LLM co-designer Workbench Screenshot-1

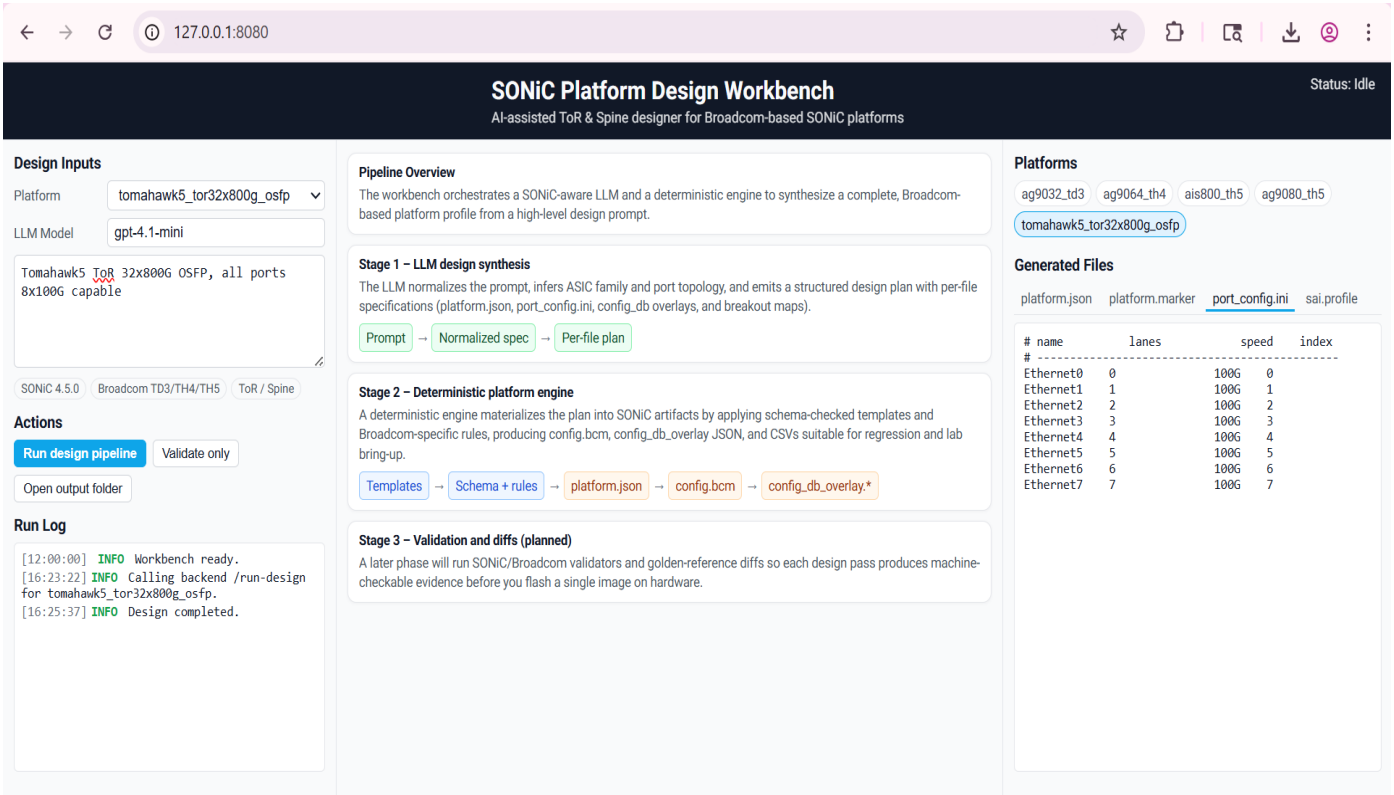

Figure-4: Agentic LLM co-designer Workbench Screenshot-2

The PoC demonstrates this flow across multiple synthetic platform patterns (for example, 100G, 400G, and 800G breakout mappings and corresponding pinmaps), confirming that the same orchestrator logic can support diverse topologies by changing only the prompt and data selection. In each case, the orchestrator produces SONiC-ready output artifacts including platform.json, port_config.ini, sai.profile, and platform.marker that align directly with the file structures expected in a SONiC platform porting workflow, without any changes to the underlying pipeline.

GitHub Repo

This repository is intended as a reference implementation of the pattern, not as a production-ready tool.

Repository: https://github.com/gnalpha341/agentic-llm-switch-codesigner

A prototype implementation of this co-designer and workbench is maintained in a private GitHub repository. Screenshots in this whitepaper use realistic platform-style names, and the underlying PoC code uses fully synthetic, hand-crafted datasets consistent with that naming. The repository is kept private to avoid any confusion with commercial products or NDA-protected materials; readers interested in exploring the PoC are welcome to contact the author directly to discuss access. The concepts and patterns described here are intended to be adapted and extended by organizations with their own proprietary datasets and SDKs.

From Abstractions to Code Templates

The current PoC focuses on generating platform abstraction that can feed into existing NOS porting processes. However, the same model can drive code templates for control-plane and hardware-abstraction-layer (HAL) components.
Given a validated platform model, an agent can generate:

- Skeleton functions in C or another language that configure ports via a generic SDK-style interface.
- Templates for basic QoS, queue, and buffer profiles aligned with AI-fabric or data center usage.
- Optional hooks for enabling features such as RDMA and VXLAN, where the platform and NOS support them.

In the PoC repository, this concept is illustrated with example C snippets placed under a dedicated directory (for example, codegen/). These examples are clearly marked as illustrative and not tied to any specific SDK implementation. The intention is that such code

templates would always be reviewed and refined by engineers, rather than deployed directly to production systems.

Future directions

Several natural extensions of this work exist:

- Integration with real datasets: Enterprises and NOS vendors can replace synthetic CSVs with internal representations generated from real datasheets, design exports, and existing platform repositories, under appropriate licensing.
- Deeper SONiC and NOS integration: The platform abstraction can be mapped directly into SONiC build trees or other NOS configurations, generating skeleton platform directories, device profiles, and port configuration artifacts consistent with official porting guidance.
- Expanded code generation: Extend the code template approach described in "From Abstractions to Code Templates" to cover initial HAL glue, telemetry hooks, and test harnesses, always keeping an engineer-in-the-loop model.

Conclusion

Manual platforms bring-up for open networking NOSes is a repeatable, high-value, but often tedious task that combines deep domain knowledge with repetitive translation from documents to configuration. This whitepaper presents a practical pattern for using an agentic LLM as a platform co-designer, supported by tools and structured data, to reduce friction in that process.

The proof-of-concept demonstrates that:

- Starting from a natural-language platform description, an agent can build a consistent platform abstraction from synthetic datasets.
- The same pipeline can support multiple platform types by changing only the inputs, not the orchestrator logic.
- These abstractions can feed existing porting flows and form the basis for future configuration and code templates.

The intent is for this pattern and PoC to serve as a starting point for enterprises, NOS vendors, and open networking communities interested in applying LLMs and agentic workflows platform bring-up and related challenges.

Audience, Availability

This whitepaper is intended for:

- Network OS and platform teams (for example, SONiC distributors and similar NOS providers).
- ASIC and switch platform engineering teams.
- Open networking and AI-fabric architects who experience the pain of repeated platform bring-up.

The work is presented as a proof-of-concept and design pattern that can be extended and industrialized by organizations with access to proprietary datasets and licensing.
The proof-of-concept code, synthetic datasets, and architecture diagram are available in a private GitHub repository maintained by the author.

Trademarks and vendor references

This work targets architectures similar to high-speed merchant-silicon switches and commercial SONiC-based distributions, as described in publicly available documentation. It uses synthetic datasets inspired by public information about open networking data center switches to illustrate the approach; all example platforms are illustrative and are not intended to reproduce any specific commercial product.
The project is not affiliated with, sponsored, or endorsed by any ASIC vendor, NOS provider, or hardware manufacturer mentioned for context. All product names, logos, and brands are the property of their respective owners, and any such names are used for identification purposes only, without implying endorsement or partnership.